A New Curriculum

Author: Ludwig Alvarado Becerra

Universities often need to update their academic curricula. At the University of Typing and Drinking Energy Drinks Overnight (UTADEO), the Systems Engineering curriculum is represented as a singly linked list of courses.

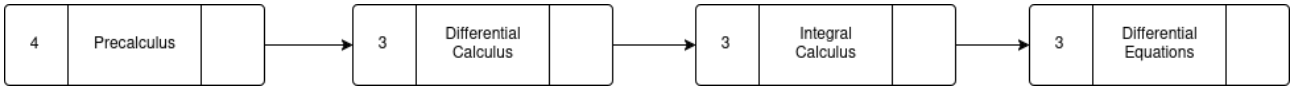

Figure 1: Calculus and mathematics line in UTADEO

Each course contains three fields:

- `Name`: the course title
- `Credits`: number of academic credits
- `Link`: pointer to the next course in the sequence

The `Link` field also indicates a prerequisite relationship: if course `A` links to course `B`, then `A` must be completed before taking `B`.

For example, in figure 1:

- `Name` = Precalculus
- `Credits` = 4
- `Link` -> Differential Calculus

This means that Precalculus is a prerequisite for Differential Calculus.

Unfortunately, during the last curriculum update, some administrators accidentally stored the course list in reverse order. As a result, students may be asked to take advanced courses before introductory ones.

Your task is to restore the correct prerequisite order by reversing the incorrectly ordered linked list.

Input

The judge provides a curriculum already loaded in memory as a singly linked list. You are given access to its first node, which is an instance of class `Node`. Each node stores three private attributes:

- `Name`: course title
- `Credits`: number of academic credits
- `Link`: reference to the next course in the sequence

Direct access to these attributes is forbidden. All interaction with the structure must be performed through the provided getters and setters:

- `set_name(new_name: str)` Updates the current course title.

- `get_name()` Returns the current course title.
- `set_credits(new_credits: int)` Updates the credits of the current course.
- `get_credits()` Returns the credits of the current course.
- `set_next(next_course: Node)` Redirects the current link to another node.
- `get_next()` Returns the next node in the sequence, or `None` if the node is the tail.

The list contains exactly N nodes, where:

$$2 \leq N \leq 10^7$$

The courses currently appear in the wrong order. The first provided node corresponds to the course that should appear last in the corrected curriculum.

No additional guarantees are given about course names or credits beyond being valid values.

Output

Reconstruct the curriculum so that prerequisite relations are restored and the sequence becomes academically consistent.

After modifying the structure, the final curriculum must be shown using the public method:

`print_list()`

This method prints the nodes from the current head following their `Link` references until the end of the list.

Your solution is expected to transform the existing linked structure itself rather than rebuilding an unrelated one.

Any output not generated through the final valid structure may be ignored by the judge.

Note

You don't have to worry about the implementation of all the functions mentioned, assume they are already working properly.

Sample Cases

Test #1

Input	Output
`Differential Equations`	`Precalculus -> Differential Calculus -> Integral Calculus -> Differential Equations`

Important Aspects

The following points describe how this exam problem will be evaluated and what restrictions must be considered during the solution process:

1. The test will be solved in **two stages**:

   - Design and explanation of the algorithm on paper **(3.0 points)**.
   - Correct implementation in the code editor **(2.0 points)**.

   Both stages are graded independently. A correct idea with poor implementation, or correct code with no reasoning, may lose points.

2. This exercise evaluates topics studied up to **singly linked lists**, including:

   - Traversal
   - Pointer/reference manipulation
   - Node relinking
   - Iterative control structures
   - Stacks and queues.

3. Since the list may contain up to 10^7 nodes, your solution must be efficient.

   Algorithms with repeated traversals, unnecessary copying, or quadratic behavior may fail practical evaluation.

4. The intended solution modifies the existing structure by changing links between nodes.

5. You are expected to use only the methods already provided in the statement:

   - `get_next()`
   - `set_next(...)`
   - and the remaining getters/setters when needed

   Direct access to private attributes is considered incorrect.

6. No external libraries are required for this problem.

   If your solution depends on advanced modules or imported data structures, it is likely not the intended approach.

7. You may assume that the initial linked list is valid:

   - It contains exactly N nodes.
   - The final node points to `None`.
   - There are no broken references.

   Therefore, no defensive validation is necessary.

8. The final output must be generated only through:

   `print_list()`

   Manual printing of names or custom formatting does not replace the required final structure.

9. Output format must match exactly the judge expectation.

   Any missing links, extra spaces, duplicated nodes, infinite loops, or incorrect termination will be judged as wrong answer.